ESP32 Wireless Mesh Networking Guide

(Absolute Beginner Step-by-Step Tutorial)

Overview

This guide walks you through setting up a peer-to-peer **ESP-MESH** network across multiple ESP32 boards using the **painlessMesh** library. Unlike traditional Wi-Fi setups, mesh nodes communicate directly with each other without needing a central Wi-Fi router.

Prerequisites & Hardware Needed

- **At least 2x ESP32 Microcontroller Boards** (e.g., ESP32 Dev Module / NodeMCU)
- **Micro-USB or USB-C Data Cables** (Ensure they support data transfer, not power-only)
- **Windows/Mac PC** with Arduino IDE 2.x installed

Step 1: Install Required Libraries in Arduino IDE

1. Open Arduino IDE and go to Tools > Manage Libraries... (or press Ctrl + Shift + I).
2. In the search box, type **painlessMesh**.
3. Locate **painlessMesh** by *guerra / spaniak* and click **Install**.
4. When prompted to install missing dependencies (such as **ArduinoJson**, **TaskScheduler**, and **AsyncTCP**), click **Install All**.

Step 2: ESP-Mesh Broadcast Code

1. Create a new sketch in Arduino IDE (File > New Sketch).
2. Replace all existing code with the following snippet:

[Use the file of ESP_MESH.ino]

Step 3: Upload Code to Multiple ESP32 Boards

1. Plug in your **first ESP32 board** via USB.
2. Select your board under Tools > Board > esp32 > ESP32 Dev Module.
3. Select the active COM port under Tools > Port.
4. Click the **Upload** button.
5. Once uploaded, **unplug Node 1**, plug in **Node 2**, select its COM port, and upload the **exact same code**.

(Note: painlessMesh automatically generates a unique Node ID for each board based on its hardware MAC address.)

Step 4: Verify Mesh Communication

1. Power both ESP32 boards simultaneously (via USB power bank, wall adapter, or PC USB ports).
2. Open the **Serial Monitor** (Tools > Serial Monitor or Ctrl + Shift + M) for one of the

connected boards.
3. Set the baud rate dropdown to **115200 baud**.
4. You will see initial network setup logs followed by messages received from nearby mesh nodes every 3 seconds:
   --> New Connection, nodeId = 349182041
   Received from 349182041: Hello from Node: 349182041

Common Mistakes & Troubleshooting

- **1. Using delay() in loop():**
  - *Mistake:* Adding standard delay(1000) calls inside loop().
  - *Why it breaks:* delay() freezes execution, preventing mesh.update() from servicing network packets. This leads to dropped connections and mesh instability.
  - *Fix:* Use TaskScheduler (as shown in the code) or non-blocking millis() timing loops.
- **2. ArduinoJson Version Mismatch:**
  - *Mistake:* Upgrading to **ArduinoJson v7.x** when painlessMesh expects **v6.x**.
  - *Why it breaks:* Breaking API changes in ArduinoJson cause build/compilation errors.
  - *Fix:* In Library Manager, lock ArduinoJson to **version 6.21.x**.
- **3. Mismatched Credentials Across Nodes:**
  - *Mistake:* Typos or differences in MESH_PREFIX, MESH_PASSWORD, or MESH_PORT between boards.
  - *Fix:* Ensure all variables match character-for-character across every board in the network.
- **4. Power Supply Brownouts & Resets:**
  - *Mistake:* Running multiple nodes off low-current USB hubs or low-quality micro-USB cables.
  - *Why it breaks:* ESP32 Wi-Fi RF transmissions create current spikes (>400mA). Underpowered supplies cause random board resets.
  - *Fix:* Use reliable power sources and consider placing a 10µF–100µF capacitor across the 3V3 and GND pins on custom breadboard setups.
- **5. Wi-Fi Channel Conflicts:**
  - *Mistake:* Forcing different channel configurations when attempting to bridge the mesh to a local home Wi-Fi router.
  - *Fix:* All mesh nodes must operate on the same Wi-Fi channel as the gateway router if bridging is enabled.